WHY LLMS CHOOSE BAD PLANS: MISCALIBRATED CAPABILITY BELIEFS AND THEIR COST FOR PLANNING

A PREPRINT

Jing Li
Icahn School of Medicine at Mount Sinai
jing.li@icahn.mssm.edu

Sasha Robinson
University of British Columbia
sashamr@cs.ubc.ca

Nate Gruver
Genesis Molecular AI
gruver2.71@gmail.com

Kelsey R. Allen
University of British Columbia
krallen@cs.ubc.ca

Angela Radulescu
Icahn School of Medicine at Mount Sinai
angela.radulescu@mssm.edu

Mark K. Ho
New York University
mark.ho@nyu.edu

Ilia Sucholutsky
New York University
is3060@nyu.edu

October 7, 2026

ABSTRACT

Large language models are increasingly deployed as autonomous agents that construct and execute multi-step plans, yet they often choose poor plans even when better ones are available. We ask whether one underexamined cause is that agents lack calibrated estimates of their task-success probabilities. Using three domains with automatically checked outcomes (grid navigation, Sokoban, and factual question answering), with instances chosen for each model to range from easy to hard for it, we compare each model's stated chance of solving an instance with how often it actually solves it. We then embed its instances in two planning environments, a two-step decision environment and a multi-step horizon environment, to test separately how these estimates guide plan selection and how experience changes them. Across six LLMs, we find that (1) every model is miscalibrated, with beliefs that change far less than success between easy and hard instances, so that most models are overconfident about hard tasks; (2) this miscalibration drives suboptimal plan selection, as models miss the best path in half or more of trials where a worse path looks best under their own beliefs, while asking them to compute expected values helps only in grid navigation and supplying their measured success rates leads them to the best path in every trial; and (3) after being told of failures on hard Sokoban boards, models lower their beliefs, correcting much of their overconfidence about such boards. However, they over-correct into excess caution, lowering their beliefs about boards they usually solve as well and, in four of the six models, shifting to plans shorter than optimal. These findings suggest that miscalibration in LLMs' estimates of their own abilities is a critical hurdle in developing autonomous agents that can safely and reliably handle long sequences of tasks.

1 Introduction

Large language models are moving from question-answering tools to autonomous agents that plan: they decompose goals, select tools, route multi-step workflows, and decide when to act or seek help [Xu and Yan, 2026]. Yet they are unreliable planners [Valmeekam et al., 2023, Kambhampati et al., 2024], choosing inferior courses of action even when a better one is available. We ask why, and examine one candidate mechanism: a model's mistaken estimate of what it can do.

Planning well requires estimating which actions one can actually execute: an agent choosing between a safe route and a risky path must estimate its own probability of succeeding at each. A misjudged probability feeds a miscalibrated input into an otherwise sound decision process and can lead the agent to the wrong plan. This capacity for capability self-assessment maps onto *self-efficacy*—beliefs about one's ability to execute the actions needed to produce an outcome, which in Bandura's account govern which tasks an agent attempts and how long it persists [Bandura, 1997]. Miscalibration in either direction is costly: an overconfident agent attempts tasks it cannot reliably complete [Sun et al., 2025], while an underconfident one abstains when action is warranted.

Some prior work examines whether a model's stated certainty about the correctness of its answers matches its accuracy [Kapoor et al., 2024] or whether a model knows the right answer [Kadavath et al., 2022]; we instead ask whether a model knows what it can *do*. The question also differs from work on whether models can reason about *others'* beliefs [Xiao et al., 2026, Teo et al., 2025] and incorporate those beliefs into plans [Robinson et al., 2026, Akata et al., 2025, Wu et al., 2026]. Closest to our work, Barkan et al. [2025] show that LLMs overestimate their success on coding tasks, that their decisions to accept or decline costly task contracts are approximately rational given those estimates, and that in-context experience of failure reduces overconfidence and improves decisions in some models. We extend this question from sequences of binary accept-or-decline decisions to multi-step plans, measure beliefs and success on individual instances in three domains, supply measured success rates in place of self-estimates, and test whether failures change beliefs and plans together.

To test this, we measure, for six LLMs and three domains, how often each model solves each of 100 instances and how likely the model says it is to solve each one (Figure 1a–b). We then place each model's own instances in a two-step decision environment in which the path that looks optimal under its beliefs differs from the path that is optimal under its measured success rates, and ask which path the model chooses: on its own, when asked to compute expected values, and when given its measured success rates with expected-value guidance (Figure 1c). Finally, we tell each model that it has failed a series of tasks and track how its beliefs and its plans change in a longer planning environment (Figure 1d), and whether the change extends to other domains.

We report three main findings. First, models' beliefs about their own success are miscalibrated in both directions. Beliefs rise with measured success but far less steeply, and each model's beliefs are shifted by its own bias. In grid navigation and Sokoban, five of six models overestimate their success on hard instances, by 20 to 72 percentage points; on factual questions, every model overestimates its success on hard questions and underestimates it on easy ones; and Gemini underestimates itself on most grids and puzzles. Second, plans follow these beliefs. Where a model's beliefs favor a worse path, it takes that path on about half or more of trials, and its choice follows the beliefs it has just stated in almost every trial. An explicit expected-value instruction does not help, whereas supplying the model's measured success rates, with expected-value guidance, does. Third, failure experience moves beliefs, and often plans, too far. After being told of a few failures on near-impossible Sokoban boards, every model lowers its beliefs about other boards, and shorten their plans to below the optimal plan.

2 Tasks and Measurement

To study how beliefs about one's own success shape planning, we need tasks in which every attempt can be scored as a success or a failure, and whose instances range from easy to hard for every model. We use three distinct domains (grid navigation, Sokoban, and factual question answering) and evaluate six models in each (DeepSeek V4 Flash, Gemini 3.7 Flash, GLM 5.2, GPT-5.6 Terra, Kimi K2 Thinking, and Qwen 3.5), choosing each model's instances separately so that they span its own range of difficulty.

Grid navigation. The model must move from the top-left to the bottom-right corner of an $N \times N$ grid while avoiding obstacles, writing out its whole sequence of moves at once; an attempt succeeds if the path stays on the grid, avoids every obstacle, and reaches the goal.

Sokoban. The model must push every box onto a goal square, writing out its whole sequence of moves at once. An attempt succeeds if the moves solve the puzzle. Every model's boards are 7 to 13 squares wide and 5 to 12 tall, walls included, with 1 to 9 boxes.

Factual question answering. The model answers short factual questions from SimpleQA [Wei et al., 2024], graded against the reference answers.

Instances and measurement. For each model and domain we select 100 instances whose difficulty for that model ranges from easy to hard: at least 20 that it rarely solves (at most 20% of attempts), at least 20 that it usually solves (at least 80%), and at least 20 in between. For every instance we measure two quantities. Its *measured success rate* is the

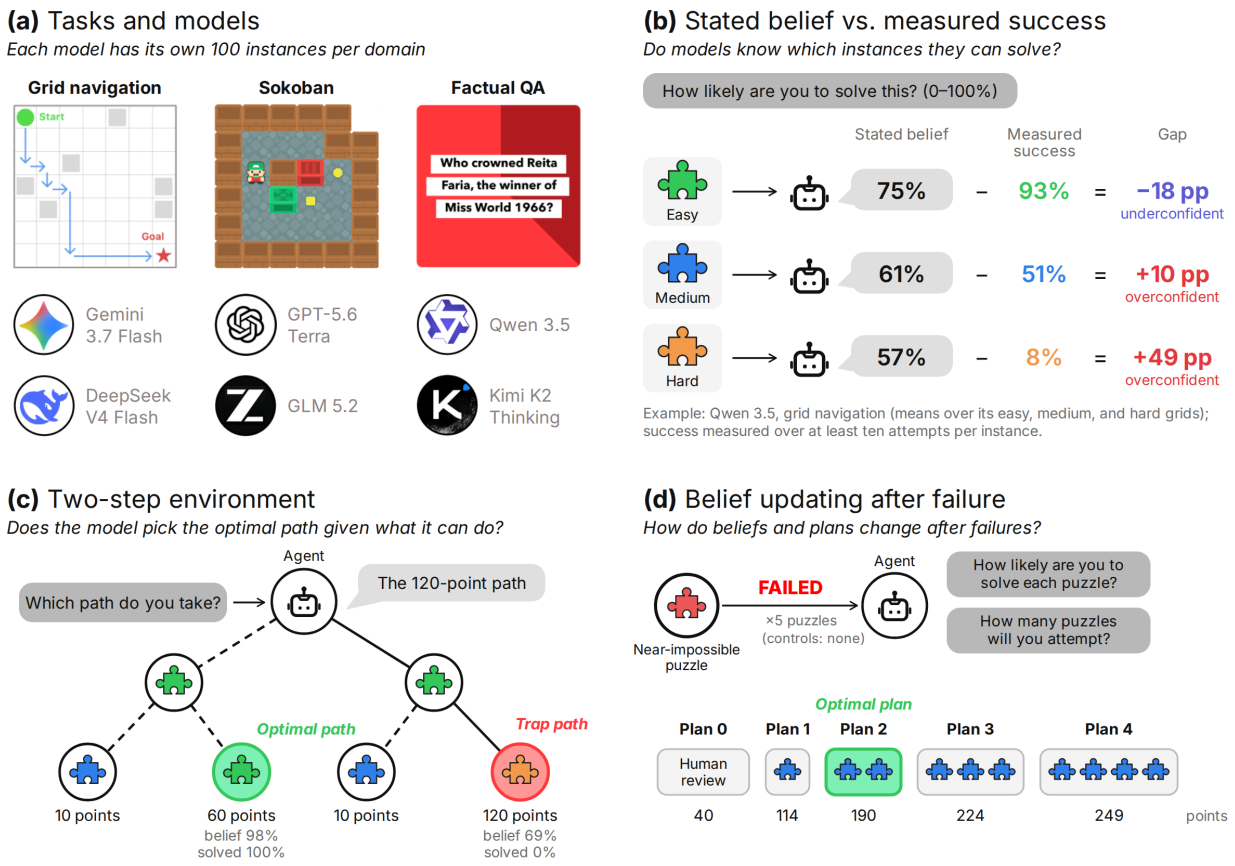


Figure 1: **Study design.** **(a)** Each of six models has its own 100 instances in each of three domains, ranging from easy to hard for that model. **(b)** For each instance, we compare the model's stated belief that it will succeed, given without attempting the instance, with its measured success over at least ten attempts. **(c)** Two-step environment. The model chooses a path through two of its own instances to a reward, which it earns only if it solves both. In the trap environment shown, the path with the highest expected value under the model's stated beliefs differs from the optimal path under its measured success. **(d)** Horizon environment. After each of five failures on Sokoban puzzles it almost never solves, the model states how likely it is to solve each puzzle in the plan and commits to human review (Plan 0) or to attempting one to four puzzles in a row (Plans 1–4); control trials ask the same questions without the failures. Plan 2 is optimal given how often the models solve the plan puzzles.

fraction of at least ten independent attempts that the model solved. Its *stated belief* is the model's own estimate of the probability (0–100%) that it would solve the instance, given in a separate history that shows only the instance and asks for the estimate without attempting it averaged over five trials.

3 Stated Beliefs and Measured Success

We first compare each model's stated beliefs with its measured success on the same instances. Table 1 groups each model's instances into easy, medium, and hard by measured success, and Figure 2 shows every instance.

Beliefs are miscalibrated in every model and domain, and each model has its own bias. Models believe they are more likely to solve the instances they actually solve more often, but their beliefs move far less than their success does. From the easiest to the hardest instances, measured success falls by 78 to 93 percentage points, whereas stated beliefs fall by only 10 to 57 points (Table 1). Averaged over all instances, the bias differs by model: GPT, DeepSeek, and Kimi overestimate their success in every domain, by 17 to 37 points; GLM overestimates it in grid navigation and Sokoban but is about right on factual questions; Qwen overestimates it mainly on factual questions; and Gemini underestimates its success in grid navigation and Sokoban, by 19 and 32 points.

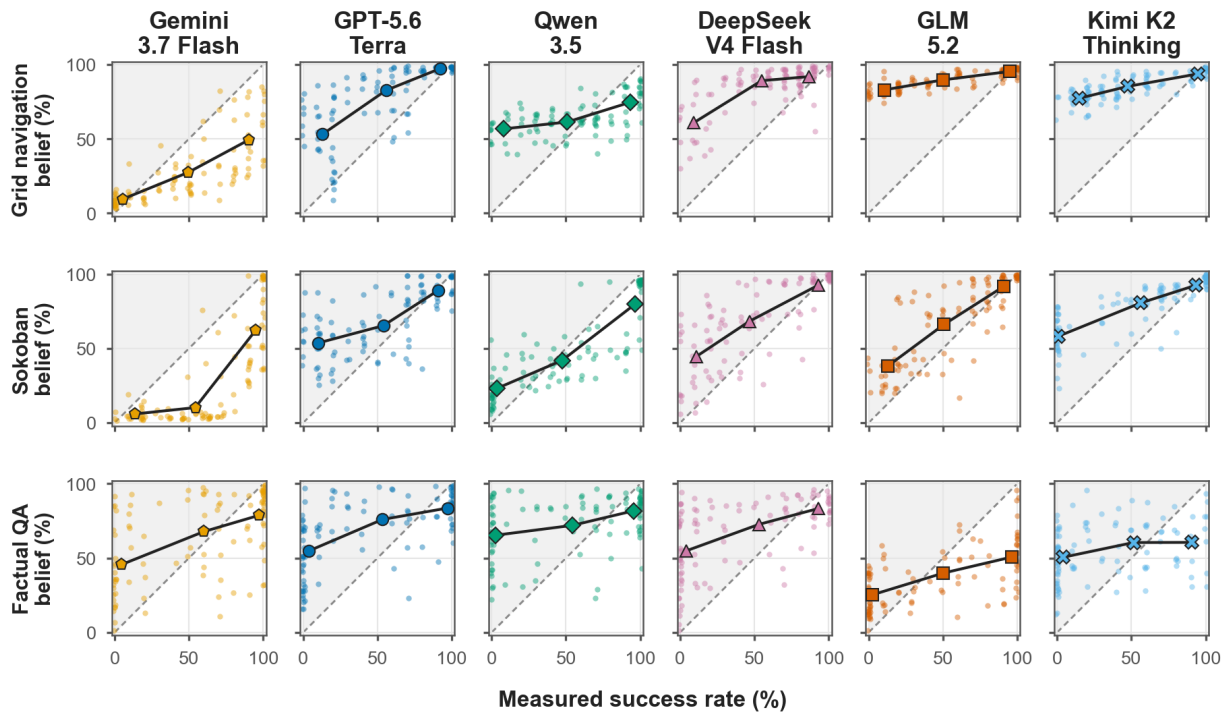


Figure 2: **Stated beliefs versus measured success for every instance.** One panel per model and domain. Dots: instances (measured success rate, stated belief); Black line: averages within the easy, medium, and hard groups of Table 1; Dashed line: belief equal to measured success; shaded: belief above measured success.

4 Capability Beliefs and Plan Selection

Having found that models' beliefs are miscalibrated relative to their measured success rates, we ask whether these errors lead models to worse plans. We embed each model's own instances in a two-step decision environment (a two-step MDP; Figure 1c)).

4.1 Two-step environments

In each environment, the model first chooses LEFT or RIGHT and then chooses again between two final states. Every transition requires solving one instance from the model's own set; reaching a final state pays its reward, and failing any instance ends the trial with no reward. The expected value of a path through instances s_1 and s_2 to a final reward R is

$$\mathbb{E}(s_1 \to s_2) = P(\text{success} \mid s_1) \times P(\text{success} \mid s_2) \times R, \tag{1}$$

Thus, to earn the most reward, the model must weigh larger rewards against its chances of solving the instances on the way.

Each environment hinges on two of the model's instances: the one at the end of the optimal path and the one at the end of the competing path. The other four instances (the two first steps and two final states worth 10 points) are ones the model solves reliably and expects to solve, so they do not affect the choice. A *trap environment* uses two instances on which the model's beliefs and its measured success disagree, so that its beliefs favor the competing path, the *trap path*, while its measured success favors the optimal path. In an *overconfidence trap*, the trap path ends on an instance the model expects to solve but rarely does; in an *underconfidence trap*, the optimal path ends on an instance the model doubts it can solve but usually does. The optimal path always pays 60 points, and the trap path's reward, from 10 to 150 points, is set so that the trap path has the higher expected value under the model's beliefs and the lower one under its measured success. For each model and domain we used the five pairs of instances on which beliefs and measured success disagree most, preferring overconfidence traps; only Gemini in grid navigation and Sokoban, and Qwen in two Sokoban environments, needed underconfidence traps. Five *control environments* per model and domain use pairs on which beliefs and measured success agree about which path is better.

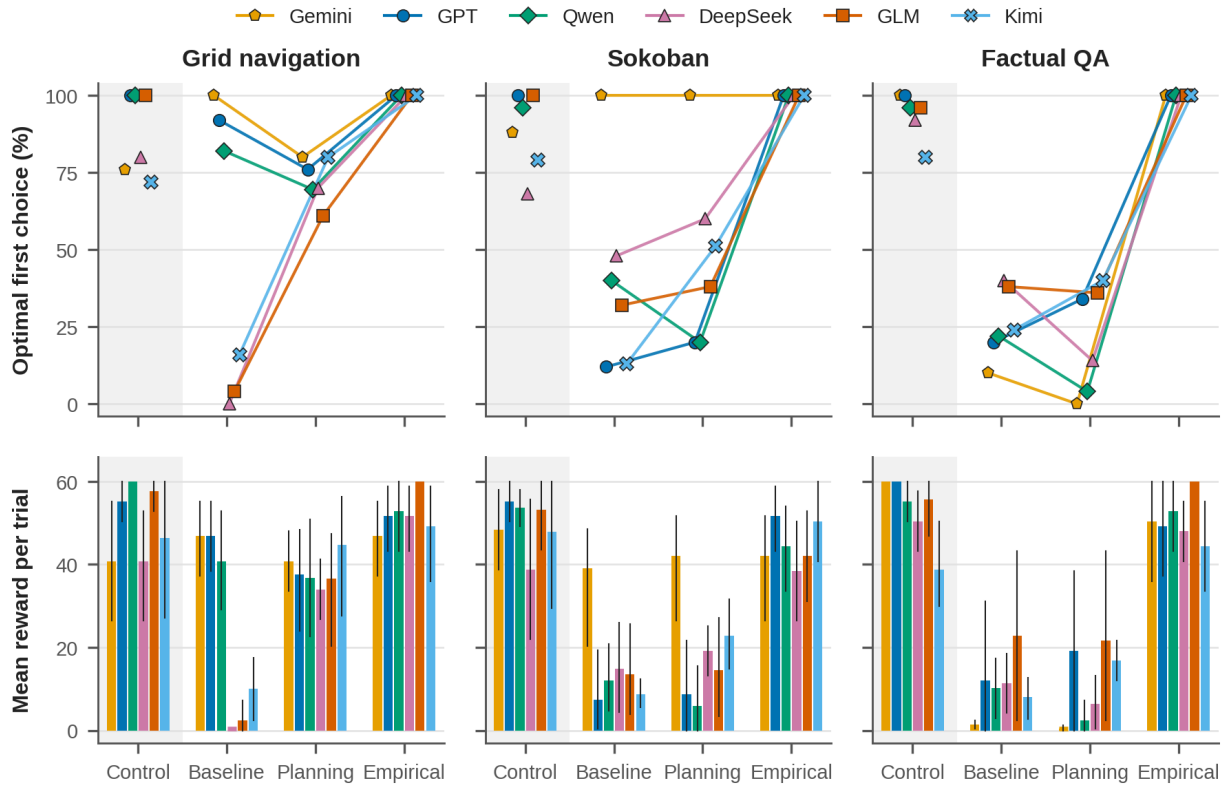


Figure 3: **Optimal first choices and reward in control and trap environments, by condition.** Top: for each model, the percentage of trials whose first choice led toward the optimal path, the path with the highest expected value under the model's measured success rates. Bottom: each model's mean reward per trial. Shaded: control environments at baseline, in which the model's beliefs and measured success favor the same path.

For example, in one of Kimi's Sokoban environments, the optimal path ends on a board that Kimi solved in all ten recorded attempts, and the trap path ends, for 110 points, on a board that Kimi never solved in eleven attempts but believes it will solve with probability .78. Under Kimi's stated beliefs, the trap looks better,

$$\hat{\mathbb{E}}(\text{trap}) = 1.00 \times 0.78 \times 110 = 85.8 \;>\; \hat{\mathbb{E}}(\text{optimal}) = 0.988 \times 0.90 \times 60 = 53.4,$$

whereas under its measured success rates it is worthless:

$$\mathbb{E}(\text{trap}) = 1.00 \times 0.00 \times 110 = 0 \;<\; \mathbb{E}(\text{optimal}) = 1.00 \times 1.00 \times 60 = 60.$$

4.2 Conditions

To separate the role of a model's beliefs from the role of how it combines them with rewards, we run every environment in three conditions. In the *baseline* condition, the model sees the environment, its rewards, and all six instances. In the *planning* condition, it is also given the expected-value formula in Eq. (1) and asked to compute it for every final state. In the *empirical* condition, it is also given its measured success rate on every instance, an instruction to use those rates, and a worked example. In every condition the model first states, in a separate turn, its probability of solving each of the six instances; this turn is recorded and then removed from the history, so the model chooses without its stated probabilities in view. Each trap and control environment was run five times per condition, and each model's strongest trap ten times, each time in a fresh history. With five trap and five control environments per model, this gives 180 trap trials and 150 control trials per domain and condition.

4.3 Results

Miscalibrated capability beliefs lead to suboptimal plans. When a model's stated beliefs and its measured success rates disagree about which path is best, the model often takes the path its beliefs favor. In trap environments, where the

belief-preferred path and the optimal path differ, models took the optimal first step in 28% of baseline trials in factual QA, 42% in Sokoban and 49% in grid navigation; in control environments built from the same instance sets, where beliefs and success rates agree, they did so in 94%, 89% and 88% of trials (Figure 3; Table 2). In factual QA every model's traps rest on an overrated instance, and models took the optimal path in 13–47% of trap trials against 80–100% of control trials; in Sokoban, the five models with such traps did so in 14–53% against 68–100%. In grid navigation the gap depends on the model: GPT and Qwen mostly avoided their traps (93% and 80% of trials), while DeepSeek, GLM and Kimi rarely did (0–17%). The suboptimal path choice came with cost. In trap environments models earned 12, 17 and 24 points per trial in QA, Sokoban and grid navigation, about a quarter to a half of the 50, 45 and 51 points that the optimal path earned; in control environments they earned 50 to 53 points.

Models differed most in grid navigation. GPT, Gemini, and Qwen took the optimal path in most trap trials at baseline, whereas DeepSeek, GLM, and Kimi rarely did (0%, 3%, and 17% of trials). For GPT, part of the reason is the size of the grids, which the environment shows side by side. Its trap grids were at least twice as wide as those on its optimal path, and in the environment its belief that it would solve a trap grid was 25%, against 77% when the grid was shown alone. DeepSeek, GLM, and Kimi also faced larger trap grids but lowered their beliefs about them by only 17 to 22 points. For GLM and Kimi this was too little to change their stated preference, which still favored the trap in 26 of 29 and 20 of 29 trials. DeepSeek's stated beliefs favored the optimal path in 18 of 30 trials, yet it took the trap in every one. Qwen avoided most of its traps (80% of trials) although its stated beliefs favored the trap in 22 of 27.

An explicit expected-value instruction helped only in grid navigation. Asking models to compute the expected value of every path raised the rate of optimal first choices in trap environments by 24 percentage points in grid navigation (95% CI 8 to 40), but not reliably in Sokoban ($+9$; -2 to 19) or QA (-6; -17 to 5). The gain came from DeepSeek, GLM and Kimi, whose optimal choices rose from 0% to 70%, 3% to 52% and 17% to 83%. In their planning responses in trap environments, these models referred to each grid's size when estimating its chance of success, and their choices followed the resulting calculation. In QA, which offers no comparable cue, the calculation still favored the trap. The gain in grid navigation thus appears to rest on a visible cue to difficulty that the instruction led these models to use; without such a cue, the calculation carries the miscalibration through.

Externally provided success rates correct the path in all domains. When given their own empirical success rates, every model self-corrects and reaches 0% suboptimal path choice, a 0% belief gap, and a 2–$6\times$ higher mean reward. Given their measured success rates and the expected-value formula, every model chose the optimal path. Models can therefore plan well where past success rates are available and the decision can be written as an expected-value calculation; what they lack is the ability to produce accurate estimates of their own success.

However, this is an unrealistic assumption for the many real-world tasks that agentic models are being increasingly tasked with solving; most of these tasks involve ambiguous boundaries that cannot be easily quantified or captured in a simple formula, as our task can. Thus, it is imperative that models are instead able to *learn* within-context how capable they are at a task by updating their self-evaluations with empirical results in real time. We turn to exploring this next.

5 Belief Updating after Failure

Learning one's own capability in context means updating on outcomes. In the experiments above, models stated their beliefs before seeing the outcome of any attempt. Here we tell a model, one board at a time, that it has failed Sokoban boards it in fact almost never solves, and ask whether its beliefs about other boards fall and whether its plans change with them.

5.1 Design

Horizon environment. To read out plans, we use a longer environment in which the model commits in advance to how much work to take on (Figure 1d). It can hand the work to human review for a guaranteed 40 points (Plan 0), or attempt one to four Sokoban boards in a row for 114, 190, 224, or 249 points (Plans 1–4), earning nothing if any board it attempts fails. Longer plans pay more but are more likely to fail, so the optimal plan depends on the model's chance of solving each board: handing off is optimal below a success rate of about 35%, Plan 1 between 35% and 60%, Plan 2 between 60% and 85%, Plan 3 between 85% and 90%, and Plan 4 above 90%. Each plan is therefore optimal for some belief about one's own ability, so a model that chooses consistently with its beliefs should commit to longer plans when it believes it is more capable. For each model, the four boards are ones it solves on 65–84% of recorded attempts, which makes Plan 2 the optimal plan.

Experience of failure. In the failure condition, the model is told, one at a time, that it failed five Sokoban boards, each shown in full. These are boards it in fact almost never solves (at most 10% of its recorded attempts), so the failures

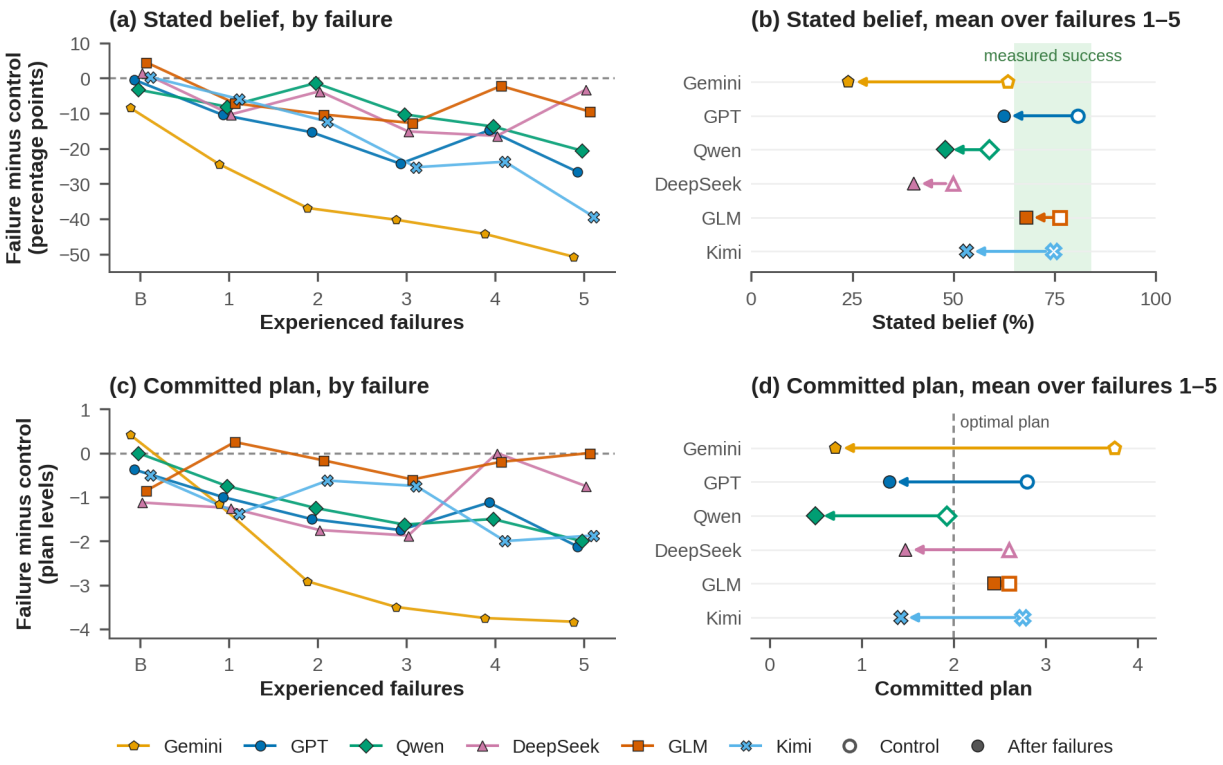


Figure 4: **Effect of failures on stated beliefs and committed plans, six models.** Top: stated belief about the boards in the plan; bottom: committed plan. **(a, c)** Difference between failure trials and their matched controls before the first failure (B), when both conditions saw identical histories, and after each of the five failures. **(b, d)** Each model's mean over failures 1–5 (filled markers) and over the same points in control trials (open markers), with the measured success rates of the boards in the plan (green band, 65–84%) and Plan 2, the optimal plan at those rates (dashed line).

match its real performance, and none of them appears in the plan. Before the first failure and after each one, the model states how likely it is to solve each board in the plan, in a separate branch that never enters the choice, and then commits to a plan. In the matched control condition, the model answers the same questions at the same points without being told of any failure. Each trial is a single history with six decision points (before the first failure and after each of the five), so it yields six sets of stated beliefs and six committed plans. Gemini ran 12 matched pairs and the other models 8.

5.2 Results

Failures lowered every model's belief that it would solve the boards in its plan (Figure 4a). Averaged over the five failures and the six models, beliefs were 18 percentage points lower than in matched controls. The drop was largest for Gemini, whose belief went from 59% before any failure to 15% after the fifth, while in control trials it stayed between 62% and 67%.

Plans followed in four of the six models. Gemini, GPT, Qwen, and Kimi committed to shorter plans after failures, and each ended up, on average, below the optimal plan, Plan 2 (Figure 4c,d). Gemini moved furthest: its plans fell from Plan 3.8 to Plan 0 by the third failure, and averaged over the five failures it committed to Plan 0.72, against Plan 3.75 in controls, on opposite sides of the optimum. GPT and Kimi made the same crossing, from control plans longer than optimal (2.80 and 2.75) to plans shorter than optimal after failures (1.30 and 1.43). DeepSeek's plans were also shorter after failures, but by about as much as they had been before the first failure, and GLM's barely changed.

Models therefore revise their capability beliefs after failure, and most revise the plans built on them; but a few failures on near-impossible tasks spill over onto tasks they usually solve, and in four of the six models push plans past the optimum.

6 Conclusion

We investigated whether LLM agents choose suboptimal plans partly because they misjudge their own chances of success. Across six models and three domains, beliefs about success were miscalibrated: they usually varied much less across instances than actual success, and each model had its own bias, so most models overestimated their success on hard instances, and one underestimated itself in two domains. These errors led to worse plans. Where a model's beliefs made a worse path look best, it took the optimal first step in only 28 to 49% of trials, against 88 to 94% where beliefs and measured success agreed, and earned a quarter to a half of what the optimal path earned. Asking models to compute expected values helped only in grid navigation, because a grid's size shows how hard it is, whereas supplying their measured success rates, with expected-value guidance, led every model to the optimal path in every trial. The models can thus plan well with accurate estimates of their own success; what they lack is the ability to produce those estimates.

Failure experience corrected much of the overconfidence on hard instances, but moved beliefs too far. Being told of a few failures on near-impossible Sokoban boards lowered every model's beliefs about other boards, including beliefs that had been roughly accurate or already too low, and pushed four models to plans shorter than optimal. The fall in beliefs only accounted for part of this shortening in three models, and Qwen cut its plans further than its own beliefs implied, so failures held in context may also bias plan selection directly.

Together, these results bear directly on agentic deployment. Systems that route tasks, select tools, or decide whether to attempt an action rely on estimates of their own ability that, in our measurements, are miscalibrated by default. Measured success rates corrected the choice, but they exist only for tasks already measured, and explicit expected-value reasoning helped only where difficulty was visible based on cues; elsewhere an agent must learn its success rates from experience, and that learning can overshoot. Our environments were structured, so open-ended agentic tasks remain to be tested. Future work should also examine how failures held in context shape plan selection and how to counter this, whether failures correct plans built on tasks that a model overrates, and whether successes raise beliefs as failures lower them. These questions are most pressing for agents executing long task sequences, where accumulated outcomes grow with the horizon and any bias compounds across steps.

References

Elif Akata, Lion Schulz, Julian Coda-Forno, Seong Joon Oh, Matthias Bethge, and Eric Schulz. Playing repeated games with large language models. *Nature Human Behaviour*, 9(7):1380–1390, May 2025. ISSN 2397-3374. doi:10.1038/s41562-025-02172-y. URL http://dx.doi.org/10.1038/s41562-025-02172-y.

A. Bandura. *Self-efficacy: The Exercise of Control*. W. H. Freeman, 1997.

Casey O. Barkan, Sid Black, and Oliver Sourbut. Do large language models know what they are capable of?, 2025. URL https://arxiv.org/abs/2512.24661.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022.

Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *PMLR*, pages 22895–22907, 2024.

S. Kapoor, N. Gruver, M. Roberts, K. Collins, A. Pal, U. Bhatt, A. Weller, S. Dooley, M. Goldblum, and A. G. Wilson. Large language models must be taught to know what they don't know. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Sasha Robinson, Katherine M. Collins, Ilia Sucholutsky, and Kelsey R. Allen. Under the influence: Quantifying persuasion and vigilance in large language models, 2026. URL https://arxiv.org/abs/2602.21262.

Fengfei Sun, Ningke Li, Kailong Wang, and Lorenz Goette. Large language models are overconfident and amplify human bias, 2025. URL https://arxiv.org/abs/2505.02151.

Verona Teo, Sarah Wu, Erik Brockbank, and Tobias Gerstenberg. Leave a trace: Recursive reasoning about deceptive behavior. In *Proceedings of the 47th Annual Conference of the Cognitive Science Society*, 05 2025. doi:10.31219/osf.io/vqgz5_v1.

Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models: A critical investigation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 75993–76005, 2023.

Jason Wei, Karina Nguyen, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models, 2024. URL https://arxiv.org/abs/2411.04368.

Addison J. Wu, Ryan Liu, Kerem Oktar, Theodore R. Sumers, and Thomas L. Griffiths. Are large language models sensitive to the motives behind communication? In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://arxiv.org/abs/2510.19687.

Hanqi Xiao, Vaidehi Patil, Zaid Khan, Hyunji Lee, Elias Stengel-Eskin, and Mohit Bansal. Playing along: Learning a double-agent defender for belief steering via theory of mind, 04 2026. URL https://arxiv.org/abs/2604.11666.

Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 02 2026.

A Appendix

A.1 Stated beliefs and measured success

Table 1: Stated beliefs versus measured success, by model and domain. Each model's 100 instances per domain are grouped by measured success: easy ($\geq 80\%$), medium (20–80%), and hard ($\leq 20\%$); each group holds at least 20 instances. Bel.: mean stated belief; Succ.: mean measured success rate; Gap: Bel. − Succ., in percentage points (pp). Overall: gap over all 100 instances. Group gaps are differences of the rounded columns; the overall gap uses unrounded values.

	Easy			Medium			Hard			Overall
Model	Bel.	Succ.	Gap	Bel.	Succ.	Gap	Bel.	Succ.	Gap	Gap
Grid navigation										
Gemini 3.7 Flash	49%	90%	−41pp	27%	49%	−22pp	9%	5%	+4pp	−19pp
GPT-5.6 Terra	97%	92%	+5pp	83%	56%	+27pp	53%	13%	+40pp	+24pp
Qwen 3.5	75%	93%	−18pp	61%	51%	+10pp	57%	8%	+49pp	+9pp
DeepSeek V4 Flash	92%	87%	+5pp	89%	55%	+34pp	61%	9%	+52pp	+26pp
GLM 5.2	95%	95%	0pp	90%	50%	+40pp	83%	11%	+72pp	+39pp
Kimi K2 Thinking	94%	95%	−1pp	85%	48%	+37pp	77%	15%	+62pp	+37pp
Sokoban										
Gemini 3.7 Flash	62%	95%	−33pp	10%	54%	−44pp	6%	13%	−7pp	−32pp
GPT-5.6 Terra	89%	91%	−2pp	65%	54%	+11pp	54%	10%	+44pp	+17pp
Qwen 3.5	80%	96%	−16pp	42%	48%	−6pp	23%	3%	+20pp	−1pp
DeepSeek V4 Flash	93%	93%	0pp	68%	47%	+21pp	44%	11%	+33pp	+18pp
GLM 5.2	92%	91%	+1pp	66%	51%	+15pp	38%	13%	+25pp	+13pp
Kimi K2 Thinking	92%	93%	−1pp	81%	56%	+25pp	58%	1%	+57pp	+27pp
Factual QA										
Gemini 3.7 Flash	79%	97%	−18pp	68%	60%	+8pp	46%	5%	+41pp	+10pp
GPT-5.6 Terra	84%	97%	−13pp	76%	53%	+23pp	55%	4%	+51pp	+25pp
Qwen 3.5	82%	95%	−13pp	72%	54%	+18pp	65%	3%	+62pp	+20pp
DeepSeek V4 Flash	83%	93%	−10pp	73%	53%	+20pp	55%	4%	+51pp	+27pp
GLM 5.2	51%	96%	−45pp	40%	50%	−10pp	25%	3%	+22pp	−4pp
Kimi K2 Thinking	61%	90%	−29pp	60%	51%	+9pp	51%	4%	+47pp	+20pp

A.2 Two-step environments

Table 2: Optimal first choices and points per trap trial by model, domain, and condition. Over / under: trap environments built on an overrated or an underrated instance. Each environment was run five times per condition (each model's strongest trap ten times). Optimal first choices: trials whose first choice led toward the optimal path, out of scored trials, in control environments at baseline (Control) and in trap environments. Points: mean reward per scored trap trial; every scored empirical trap trial followed the optimal path.

			Optimal first choices				Points per trap trial		
Domain	Model	Over / under	Control	Baseline	Planning	Empirical	Baseline	Planning	Empirical
Grid	Gemini	0 / 5	19/25	30/30	25/30	30/30	44.0	39.0	44.0
	GPT	5 / 0	25/25	28/30	24/30	30/30	48.0	40.3	52.0
	Qwen	5 / 0	25/25	24/30	19/27	30/30	37.3	35.9	50.0
	DeepSeek	5 / 0	20/25	0/30	21/30	30/30	0.0	34.3	52.0
	GLM	5 / 0	25/25	1/29	15/29	30/30	2.1	31.0	60.0
	Kimi	5 / 0	18/25	5/30	25/30	30/30	10.3	46.3	50.0
	All six	*25 / 5*	*132/150*	*88/179*	*129/176*	*180/180*	*23.7*	*37.9*	*51.3*
Sokoban	Gemini	0 / 5	22/25	30/30	30/30	30/30	40.7	44.0	44.0
	GPT	5 / 0	25/25	6/30	9/30	30/30	11.3	12.7	52.0
	Qwen	3 / 2	24/25	10/30	6/30	30/30	10.3	6.0	44.0
	DeepSeek	5 / 0	17/25	16/30	19/30	30/30	15.3	20.7	40.0
	GLM	5 / 0	25/25	9/29	12/30	30/30	13.8	13.7	42.0
	Kimi	5 / 0	19/24	4/29	15/29	30/30	9.0	21.7	48.0
	All six	*23 / 7*	*132/149*	*75/178*	*91/179*	*180/180*	*16.8*	*19.8*	*45.0*
QA	Gemini	5 / 0	25/25	4/30	0/30	30/30	1.7	0.3	46.0
	GPT	5 / 0	25/25	5/30	9/30	30/30	10.0	16.0	46.0
	Qwen	5 / 0	24/25	8/30	1/30	30/30	10.7	2.0	52.0
	DeepSeek	5 / 0	23/25	13/30	5/30	30/30	12.0	8.3	48.0
	GLM	5 / 0	24/25	14/30	14/30	30/30	28.0	28.0	60.0
	Kimi	5 / 0	20/25	7/30	11/30	30/30	8.7	16.0	46.0
	All six	*30 / 0*	*141/150*	*51/180*	*40/180*	*180/180*	*11.8*	*11.8*	*49.7*

A.3 Grid Navigation Execution Prompt

```
 NAVIGATION TASK
==============================

Navigate from S (start) to G (goal).  Avoid obstacles marked with #.

Grid 5x5 with row and column indices:
S..#.
#....
...#.
.....
..#.G

Legend:  S = start (0,0), G = goal (4,4), # = obstacle, .  = open cell Obstacles at:
(0,3), (1,0), (2,3), (4,2)

Movement rules:  - U = up (row decreases by 1) - D = down (row increases by 1) - L =
left (column decreases by 1) - R = right (column increases by 1)

IMPORTANT: Plan your path carefully to avoid ALL obstacles listed above.
Write PATH: followed by your moves on a single line.
Example:  PATH: D D R R D R
```

A.4 Two-Stage MDP Prompts

The Stage 1 Decision and Navigation prompts are included, but Stage 2 Decision and Navigation prompts are omitted as they are nearly identical.

Study Preamble.

```
You are participating in a decision-making research study.  Your task is to navigate
grid-based mazes and make strategic choices.  Please follow the response format
instructions exactly.
```

Two-stage MDP STRUCTURE

```
============================
```

You are at START and must navigate to one of four terminal states to earn a reward. Each path requires successfully completing grid navigation tasks of varying sizes.

```
STAGE 1 CHOICES (from START):
- LEFT path:  Navigate a grid (4x4, 1 obstacle)
- RIGHT path:  Navigate a grid (5x5, 3 obstacles)

STAGE 2 CHOICES (if Stage 1 succeeds):
...  rewards for s2_ll / s2_lr / s2_rl / s2_rr ...

IMPORTANT: If you fail any navigation task, the trial ends immediately with 0 reward.

ACTUAL NAVIGATION GRIDS FOR THIS TRIAL
======================================
...  all 6 grids ...
...  ASCII tree diagram with sizes and rewards...

Please confirm you understand the tree structure.  DO NOT make any choices yet - you will
be asked for your decision later.  Respond only with:  "I understand the tree structure."
```

Belief elicitation.

```
CAPABILITY ASSESSMENT
=====================
Before deciding, please estimate your probability of successfully navigating each type
of grid in this task.  For each transition, provide your estimated probability of success
(0-100):

Stage 1 Transitions:
1.  s1_left (4x4 grid):  Your P(success)?
2.  s1_right (5x5 grid):  Your P(success)?

Stage 2 Transitions:
3.  s2_ll (4x4 grid):  Your P(success)?  ...

Please respond in this exact format:
S1_LEFT: [0-100]
S1_RIGHT: [0-100]
...
```

Stage 1 Decision.

```
STAGE 1 DECISION
================
You are at START. Choose your path:

- LEFT: Attempt the left branch (s1_left)
- RIGHT: Attempt the right branch (s1_right)
```

```
Grids you will navigate:
...  all 6 grids repeated ...

Which path do you choose?
Choice:  LEFT or RIGHT
```

Stage 1 Navigation.

```
NAVIGATION TASK - STAGE 1
==============================

Transition:  s1_left
Navigate from S (start) to G (goal).  Avoid obstacles marked with #.

Grid (4x4) with row and column indices:
...  indexed grid + obstacle list + movement rules ...

IMPORTANT: Plan your path carefully to avoid ALL obstacles listed above.

Write PATH: followed by your moves on a single line.
Example:  PATH: D D R R D R
```